

XS WALLET

Especificação Técnica

Carteira HD + Atomic Swaps - Aplicação Desktop

Documento	Valor
Versão	0.2.0
Status	Fase 1 Completa - Boltz Client Production-Ready
Data	Janeiro 2026
Classificação	Especificação Técnica
Público-Alvo	Desenvolvedores, Arquitetos, Auditores
Backend	Go Core (xscore) com gRPC
Swap Provider	boltz-backend (self-hosted)

Índice

1. Sumário Executivo
2. Arquitetura do Sistema
 - 2.1 Visão Geral dos Componentes
 - 2.2 Modelo de Processos
 - 2.3 Fluxo de Dados
3. Especificação do Banco de Dados
 - 3.1 Design do Schema
 - 3.2 Definições de Tabelas
 - 3.3 Modelo de Concorrência
4. Protocolo de Atomic Swap
 - 4.1 Submarine Swap
 - 4.2 Reverse Swap
 - 4.3 Chain Swap
 - 4.4 Máquina de Estados
5. Especificações Criptográficas
 - 5.1 Derivação de Chaves (BIP39/32/84/85)
 - 5.2 Criptografia do Vault
 - 5.3 Taproot + MuSig2
6. Gerenciamento de Nodes
 - 6.1 Gerenciamento de Ciclo de Vida
 - 6.2 Verificação de Binários
7. Modelo de Segurança
8. Especificações de API
9. Roadmap de Implementação

1. Sumário Executivo

XS Wallet é uma aplicação desktop self-custody que permite atomic swaps entre Bitcoin (BTC), Liquid (L-BTC) e Lightning Network (LN) usando Taproot e boltz-backend (self-hosted) como orquestrador de swap com nossa liquidez. O sistema mantém verdadeira auto-custódia enquanto fornece swaps cross-chain e cross-layer sem exigir que usuários operem sua própria infraestrutura.

1.1 Capacidades Principais

Capacidade	Implementação	Status
Go Core (xscore)	Backend autoritativo em Go com gRPC	Implementado
Boltz HTTP Client	Retry/backoff, Retry-After, erros tipados	Production-Ready
Boltz WebSocket	Single-writer, gap-recovery, connCtx	Production-Ready
Status Normalization	Table-driven por tipo de swap	Implementado
Swap Engine	CAS/optimistic locking, idempotência	Base Implementada
HD Wallet	Derivação BIP39/32/84/85	Parcial
Encryption	Argon2id + AES-256-GCM	Projetado

Tabela 1.1: Matriz de Capacidades Principais

1.2 Princípios de Design

Auto-Custódia: Usuário controla todas as chaves privadas; nenhum servidor detém fundos ou autoridade de assinatura.

Zero Trust: Todos os dados externos (respostas boltz-backend, dados de nodes) são verificados localmente antes do uso.

Crash Recovery: Todos os estados de swap são persistidos com operações idempotentes; swaps podem ser retomados após qualquer falha.

Restauração Determinística: Swaps pendentes podem ser restaurados apenas do mnemônico usando child seeds BIP85.

2. Arquitetura do Sistema

2.1 Visão Geral dos Componentes

O sistema é estruturado com Go Core (xscore) como backend autoritativo, comunicando via gRPC com o frontend Electron. O swap orchestrator é o boltz-backend self-hosted com nossa liquidez. Esta arquitetura garante que uma falha em qualquer componente não afete os outros, e operações de swap podem continuar mesmo se a UI reiniciar.

Nota: O diagrama de arquitetura visual completo está disponível na documentação original. A arquitetura atual usa Go Core com gRPC, boltz-backend self-hosted, e 2 interfaces RPC primárias (LND gRPC + elementsd JSON-RPC).

2.2 Modelo de Processos

Processo	Função	Benefício de Isolamento
Go Core (xscore)	Swap engine, database, adapters, Boltz client	Backend autoritativo isolado
Electron (Main)	Gerenciamento de janelas, IPC, spawning	Estabilidade do core
React UI (Renderer)	Interface do usuário, interações	Sandboxed; privilégios mínimos
boltz-backend	Orquestrador de swap, nossa liquidez	Self-hosted; controle total
LND	Lightning Network daemon (gRPC)	Processo separado; autenticação macaroon
elementsd	Liquid/Elements node (JSON-RPC)	Processo separado; datadir próprio

Tabela 2.1: Modelo de Processos

2.3 Fluxo de Dados

Todas as operações iniciadas pelo usuário fluem através do gRPC para o Go Core. O backend mantém estado autoritativo no SQLite e coordena com adapters e boltz-backend. Invariante crítica: **Eventos WebSocket do boltz-backend são apenas triggers; estado on-chain/LND é sempre verificado antes de agir.**

3. Especificação do Banco de Dados

3.1 Design do Schema

O banco de dados usa SQLite com Write-Ahead Logging (WAL) para concorrência otimizada de leitura/escrita. Todas as mutações de estado de swap usam Compare-And-Swap (CAS) com locking otimista baseado em versão para prevenir race conditions sem bloquear queries.

3.1.1 Pragmas Críticos

Pragma	Valor	Justificativa
journal_mode	WAL	Leituras concorrentes durante escritas; crash recovery
synchronous	NORMAL	Balanco durabilidade/performance (seguro com WAL)
busy_timeout	5000	Espera 5s por locks; previne "database is locked"
foreign_keys	ON	Força integridade referencial
temp_store	MEMORY	Tabelas temp em RAM para performance
cache_size	-20000	~20MB page cache

Tabela 3.1: Pragmas SQLite

3.2 Definições de Tabelas

3.2.1 swaps - Estado Autoritativo do Swap

Coluna	Tipo	Constraints	Descrição
id	TEXT	PRIMARY KEY	Identificador único do swap (UUID)
kind	TEXT	CHECK(IN submarine,reverse,tor,blis)swap	Tipos de swap
env	TEXT	CHECK(IN regtest,testnet,mainnet)	Ambiente de rede
version	INTEGER	NOT NULL DEFAULT 0	Versão CAS para locking otimista
state	TEXT	CHECK(IN open,locked,...)	Estado atual da máquina de estados
locked_intent	TEXT (JSON)	NOT NULL when state != open	Snapshot imutável de quote/fees
swap_key_index	INTEGER	NOT NULL	Índice de derivação BIP85
preimage_hash_hex	TEXT	CHECK(length=64)	Hash SHA256 do preimage R
claim_pubkey_hex	TEXT	CHECK(length IN 64,66)	Chave pública de claim do usuário
refund_pubkey_hex	TEXT	CHECK(length IN 64,66)	Chave pública de refund do usuário
musig_session_id	TEXT		Identificador de sessão MuSig2
lockup_txid	TEXT	CHECK(length=64)	ID da transação de funding
lockup_amount_sat	TEXT	CHECK(GLOB [0-9]*)	Quantidade em satoshis (bigint como string)

created_at	TEXT	DEFAULT ISO8601	Timestamp de criação
updated_at	TEXT	DEFAULT ISO8601	Timestamp de última atualização

Tabela 3.2: Tabela swaps (parcial - 40+ colunas no total)

Invariante: Quando state não está em (open, failed, canceled), o campo locked_intent DEVE ser não-nulo. Isso é forçado via constraint CHECK e garante que todo swap ativo tenha um registro imutável de parâmetros acordados.

3.2.2 Tabelas de Suporte

Tabela	Chave Primária	Propósito
swap_events	seq (AUTOINCREMENT)	Log de auditoria imutável; permite replay/debug
swap_ops	(swap_id, op_key)	Ledger de operação idempotente; previne broadcasts duplicados
utxo_reservations	(chain, txid, vout)	Previne double-spend de UTXO entre swaps
ln_reservations	payment_hash_hex	Previne pagamentos LN duplicados
app_config	key	Configuração do usuário; snapshot capturado em locked_intent

Tabela 3.3: Tabelas de Suporte

3.3 Modelo de Concorrência

Todas as transições de estado usam Compare-And-Swap (CAS) com transações atômicas:

```
-- Padrão de Update CAS
UPDATE swaps
SET state = 'locked',
    version = version + 1,
    updated_at = strftime('%Y-%m-%dT%H:%M:%fZ', 'now'),
    locked_intent = ?
WHERE id = ? AND version = ? AND state = 'open'
RETURNING version, state;

-- Se RETURNING está vazio: version mismatch (modificação concorrente)
-- Aplicação deve recarregar e tentar novamente ou abortar
```

Listagem 3.1: Padrão de Update CAS

4. Atomic Swap Protocol

4.1 Submarine Swap (On-chain → Lightning)

Submarine swaps convert on-chain funds (BTC or L-BTC) to Lightning capacity. The user locks funds in a P2TR HTLC; Boltz pays the user's LN invoice and claims the HTLC using the revealed preimage.

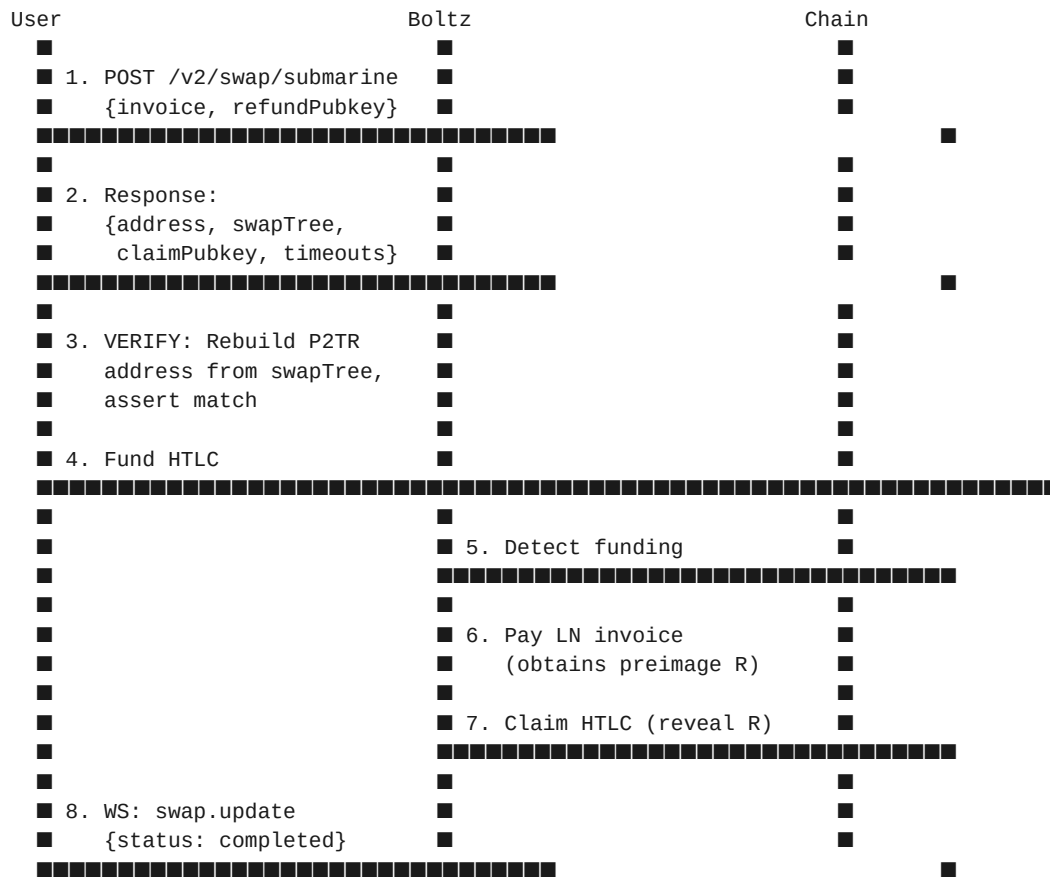
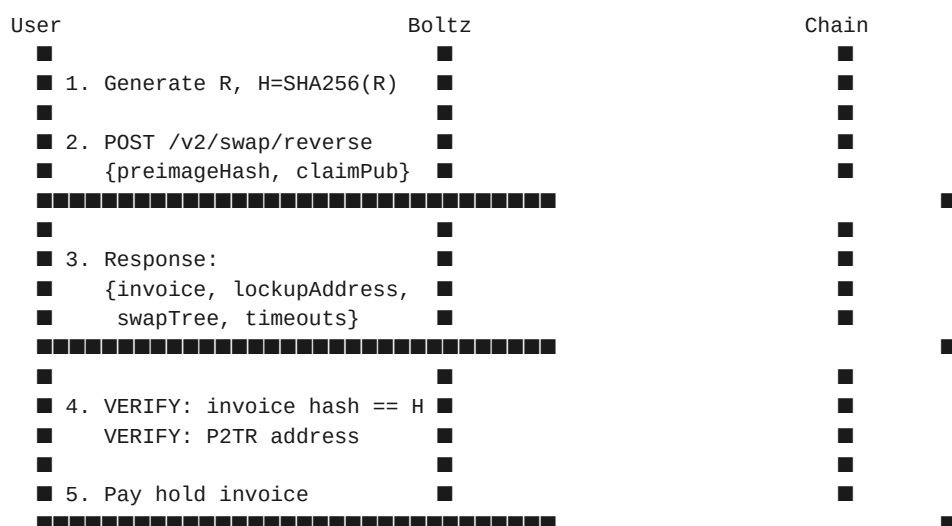


Figure 4.1: Submarine Swap Sequence

4.2 Reverse Swap (Lightning → On-chain)

Reverse swaps convert Lightning balance to on-chain funds. The user generates preimage R and hash $H = \text{SHA256}(R)$, pays a Boltz hold invoice, and claims the on-chain HTLC by revealing R.



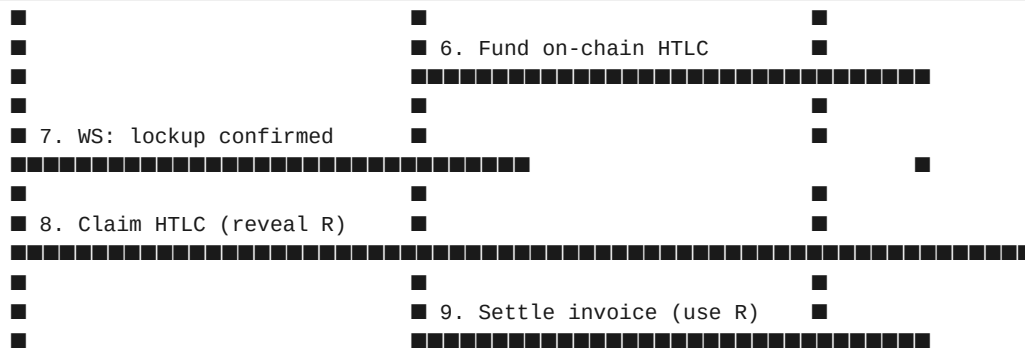


Figure 4.2: Reverse Swap Sequence

4.3 Chain Swap (BTC ↔ Liquid)

Chain swaps are atomic cross-chain swaps between Bitcoin mainchain and Liquid sidechain. Timeouts are staggered ($T_{\text{liquid}} < T_{\text{btc}}$) to prevent race conditions.

4.4 State Machine

State	Description	Transitions
open	Initial state; quote accepted but not locked	locked canceled
locked	Parameters locked; intent immutable	commit_started canceled
commit_started	Funding transaction broadcast	waiting failed
waiting	Awaiting counterparty action	waiting_claim_details refund_coop_waiting
waiting_claim_details	Awaiting claim transaction details	signing_musig2_partial
signing_musig2_partial	Creating MuSig2 partial signature	sent_partial_to_provider
sent_partial_to_provider	Partial sig sent; awaiting broadcast	waiting_provider_broadcast
waiting_provider_broadcast	Provider broadcasting claim tx	completed fallback_script_ready
refund_coop_waiting	Attempting cooperative refund	completed fallback_script_ready
fallback_script_ready	Preparing script-path claim/refund	refunding completed
refunding	Refund tx broadcast; awaiting confirmation	completed failed
completed	Terminal success state	(terminal)
failed	Terminal failure state	(terminal)
canceled	Canceled before funding	(terminal)

Table 4.1: Swap State Machine

5. Cryptographic Specifications

5.1 Key Derivation Hierarchy

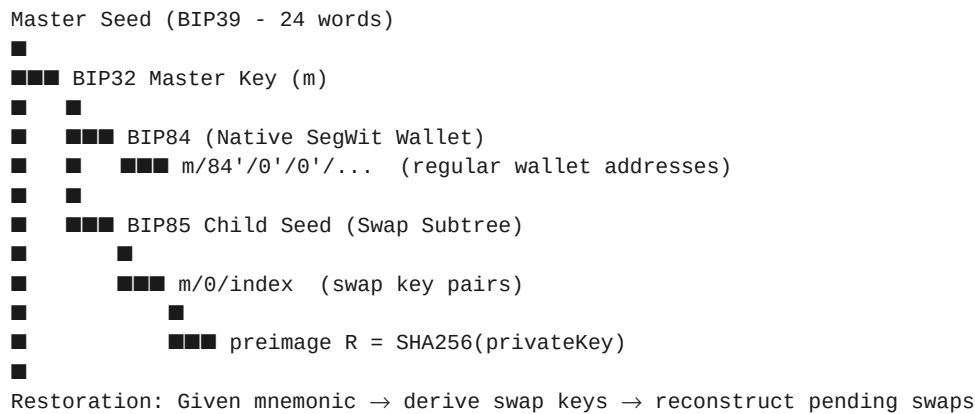


Figure 5.1: Key Derivation Hierarchy

5.2 Vault Encryption

Parameter	Value	Rationale
KDF	Argon2id	Memory-hard; resistant to GPU/ASIC attacks
Memory	64 MB	Balance security/performance for desktop
Iterations	3	Standard recommendation
Parallelism	1	Single-threaded derivation
Salt Length	16 bytes	Random per-wallet
Cipher	AES-256-GCM	Authenticated encryption
IV Length	12 bytes	GCM standard
Tag Length	16 bytes	Full authentication tag

Table 5.1: Vault Encryption Parameters

Offline Attack Mitigation: Rate limiting (exponential backoff) protects runtime but not offline database theft. For enhanced protection, derive final key from PIN + device secret stored in OS keychain (Keychain/DPAPI/SecretService). MVP uses PIN-only; device secret is v1.1.

5.3 Taproot + MuSig2

Swap HTLCs use P2TR (Pay-to-Taproot) with:

Component	Specification
Key Aggregation Order	Provider pubkey first, then user pubkey (deterministic)
Nonce Generation	Deterministic from session ID + private key

Session Persistence	musig_* fields in swaps table; survives restart
Key-Path Spend	MuSig2 aggregated signature (cooperative claim/refund)
Script-Path Fallback	HTLC conditions if cooperation fails (timeout-based)

Table 5.2: MuSig2 Specification

6. Node Management

6.1 Lifecycle Management

The Node Manager handles binary verification, spawning, health monitoring, and graceful shutdown of embedded nodes. Each node has isolated data directories per network.

Node	Version	Data Directory	Health Check
bitcoind	26.0	%APPDATA%/xs-wallet/nodes/btc/{network}/	getblockchaininfo RPC
elementsd	23.2.1	%APPDATA%/xs-wallet/nodes/liquid/{network}/	getblockchaininfo RPC
LND	0.17.3	%APPDATA%/xs-wallet/nodes/lnd/{network}/	getinfo gRPC

Table 6.1: Node Specifications

6.2 Binary Verification

Binaries are downloaded on first run from official sources (GitHub Releases, bitcoincore.org) and verified using SHA256 checksums from a signed manifest.

Trust Chain: Application embeds a pinned public key. On startup, downloads nodes-manifest.json from GitHub Releases, verifies signature against pinned key, then verifies each binary checksum against manifest. No central custody server; only public distribution endpoints.

7. Security Model

7.1 Threat Model

Threat	In Scope	Mitigation
Seed theft (offline)	Yes	Argon2id + device secret (v1.1)
PIN brute force (runtime)	Yes	Exponential backoff; lockout after 10 attempts
Swap manipulation	Yes	Local script verification; never trust provider
Supply chain attack	Yes	Signed manifest; SHA256 verification
Physical device access	No (future)	Consider hardware wallet support
OS compromise (root)	No	Out of scope; assume trusted OS

Table 7.1: Threat Model

7.2 Cryptographic Verification

Verification	When	Failure Action
P2TR address rebuild	Before funding HTLC	Abort swap; log error
Invoice hash == preimage hash	Before paying hold invoice	Abort swap; log error
Timeout sanity ($T_{\text{liquid}} < T_{\text{btc}}$)	Before locking	Abort swap; log error
MuSig2 partial signature	Before sending to provider	Abort; attempt script-path
On-chain confirmation	After claim/refund broadcast	Retry with higher fee

Table 7.2: Verification Points

8. API Specifications

8.1 Boltz API v2 Endpoints

Endpoint	Method	Purpose
/v2/swap/submarine	POST	Create submarine swap (on-chain → LN)
/v2/swap/reverse	POST	Create reverse swap (LN → on-chain)
/v2/swap/chain	POST	Create chain swap (BTC ↔ Liquid)
/v2/swap/{id}	GET	Get swap status
/v2/swap/{id}/claim	POST	Submit claim transaction details
/v2/swap/{id}/refund	POST	Submit refund transaction details
/v2/ws	WebSocket	Real-time swap status updates
/v2/nodes	GET	Get routing hints for LN payments

Table 8.1: Boltz API v2 Endpoints

8.2 Internal IPC API

Channel	Direction	Payload
swap:create	Renderer → Backend	{kind, fromAsset, toAsset, amount}
swap:status	Backend → Renderer	{swapId, state, details}
wallet:balance	Renderer → Backend	{chain: btc liquid ln}
wallet:balance:response	Backend → Renderer	{confirmed, unconfirmed, pending}
node:status	Backend → Renderer	{bitcoind, elementsd, lnd: running stopped syncing}

Table 8.2: IPC API Channels

9. Implementation Roadmap

Fase	Duração	Entregas	Status
Boltz Client	Completa	HTTP, WebSocket, Status Normalization	Concluída
Spike boltz-backend	Atual	Responsibility Split, Testes Integração	Em Progresso
Adapters	1 semana	LND gRPC, elementsd JSON-RPC	Próximo
Wiring	1 semana	xscore → boltz-backend, Testes E2E	Pendente
Frontend	2 semanas	Migrar BRLN devdash, Integração Electron	Pendente

Table 9.1: Implementation Roadmap

9.1 Post-MVP Roadmap

Version	Target	Features
v1.1	Q2 2026	Hardware wallet support, SQLCipher, device secret
v1.2	Q3 2026	Coinjoin integration, Payjoin, LNURL
v2.0	Q4 2026	RGB assets, Taproot Assets, DLC support

Table 9.2: Post-MVP Roadmap